

Backend Docs v0.1

📌 Backend Docs v0.1

Suggested Technical Architecture

Frontend

- React;
- TypeScript;
- Vite;
- Tailwind CSS;
- React Router;
- Supabase JavaScript client;
- Mapbox, Leaflet or another approved map provider.

Backend

- Supabase Auth;
- Supabase PostgreSQL;
- Supabase Realtime;
- Supabase Storage;
- Supabase Edge Functions;
- Row Level Security;
- Git and GitHub.

Recommended Backend Branches

Plain Text ▾

```
1 feature/backend/supabase-foundation
2 feature/backend/auth-verification
3 feature/backend/profiles-interests
4 feature/backend/availability
5 feature/backend/locations-map-data
6 feature/backend/activities
7 feature/backend/activity-participants
8 feature/backend/official-events
9 feature/backend/event-rsvps
10 feature/backend/matching
11 feature/backend/connections
12 feature/backend/chat-realtime
13 feature/backend/block-report
```

Suggested API and Function Contracts

Plain Text ▾

```
1 POST /functions/v1/complete-onboarding
2 POST /functions/v1/update-availability
3 POST /functions/v1/create-activity
4 POST /functions/v1/request-to-join-activity
```

```

5 POST /functions/v1/respond-to-activity-request
6 POST /functions/v1/create-official-event
7 POST /functions/v1/rsvp-to-event
8 POST /functions/v1/refresh-suggestions
9 POST /functions/v1/send-connection-request
10 POST /functions/v1/respond-to-connection-request
11 POST /functions/v1/block-user
12 POST /functions/v1/report-user

```

The frontend can read approved map data, profiles, availability, activities, events, connections and messages through the Supabase client, protected by RLS.

Sensitive actions such as matching, event approval, blocking and reporting should be handled by Edge Functions.

Core Database Tables

	Table	Purpose
1	organisations	Universities, companies and approved venue organisations
2	organisation_domains	Approved email domains
3	locations	Campuses, offices and public venues
4	areas	Libraries, food courts, buildings and workplace areas
5	profiles	User profile information
6	interests	Available interest categories
7	profile_interests	User-interest relationships
8	availabilities	User availability periods
9	activities	User-created temporary meetups
10	activity_participants	Activity join requests and participants
11	official_events	University, organisation and venue events
12	event_rsups	Event participation
13	connection_requests	Connection request status
14	connections	Accepted connections
15	messages	Chat messages
16	blocks	Block relationships
17	reports	Safety and moderation reports

Map markers can be generated from active availability, activities and events. A separate map-marker table is not required for the MVP.

Backend Objective

The backend must support the complete Recess experience:

- Plain Text ▾
- 1 Verified registration
 - 2 → Profile creation
 - 3 → Availability setup
 - 4 → Real-time map
 - 5 → User activities and official events
 - 6 → Matching
 - 7 → Connection requests
 - 8 → Private chat
 - 9 → Blocking and reporting

The backend must enforce these rules:

- Only verified students and professionals can use core features.
- Users must register with approved university or organisation emails.
- Exact personal GPS locations must never be exposed.
- Activities and events must expire automatically.
- Only authorised users can access profiles, chats and map data.
- Blocked users must not appear in suggestions, activities or map results.

Phase 0 — Backend Foundation

Suggested duration: 2–3 working days

	≡ Task	≡ Deliverable
1	Create Supabase development project	Development environment
2	Configure environment variables	<code>.env.example</code> and setup documentation
3	Set up Supabase migrations	Migration structure
4	Define naming and branch conventions	Backend contribution guide
5	Create base organisation tables	<code>organisations</code> , <code>organisation_domains</code>
6	Create location tables	<code>locations</code> , <code>areas</code>
7	Add seed data	Initial universities, companies and areas
8	Define API response format	Shared API contract
9	Configure basic testing structure	Test environment
10	Select map provider	Map provider and API key decision

Exit Criteria

- Supabase development environment is available.
- Migrations can be applied successfully.
- Initial organisations and areas are available.
- Environment variables are documented.

- Frontend team receives the first database and API documentation.

Phase 1 — Authentication and Organisation Verification

Suggested duration: 4–5 working days

	≡ Task	≡ Deliverable
1	Configure Supabase Auth	Email/password authentication
2	Implement email verification	Verified account flow
3	Validate approved email domains	Domain validation logic
4	Link users to organisations	Organisation membership
5	Create profile on registration	Automatic profile creation
6	Add account status fields	<code>pending</code> , <code>verified</code> , <code>suspended</code>
7	Restrict unverified users	Access control
8	Store terms and privacy consent	Consent records
9	Test valid and invalid domains	Authentication test cases
10	Document authentication flow	Frontend integration guide

Required Rules

- `student@uts.edu.au` may register if the domain is approved.
- `person@gmail.com` must be rejected.
- An account cannot access matching, activities or chat before verification.
- The email address must not be visible to other users.

Exit Criteria

A student or professional can register, verify their email and be linked to the correct organisation.

Phase 2 — Profiles, Interests and Availability

Suggested duration: 5–7 working days

	≡ Task	≡ Deliverable
1	Create profile schema	<code>profiles</code> table
2	Create interests schema	<code>interests</code> , <code>profile_interests</code>
3	Create availability schema	<code>availabilities</code> table
4	Add profile CRUD operations	Profile management

5	Add interest management	Interest selection support
6	Add availability validation	Valid time-range handling
7	Add approximate area selection	Area-based location
8	Add availability expiry logic	Expired records ignored
9	Implement profile RLS	User ownership protection
10	Implement availability RLS	Private availability control
11	Add indexes	Efficient queries
12	Write unit and integration tests	Test coverage

Privacy Rules

- Store `area_id`, not exact user coordinates.
- Do not expose a user's email address.
- Do not expose a user's exact location.
- Users can only edit their own profile and availability.
- Expired availability must not appear on the map or in matching.

Exit Criteria

A verified user can create a profile, select interests, set availability and select an approximate area.

Phase 3 — Real-Time Map, Activities and Events

Suggested duration: 7–10 working days

This phase is important because the real-time map and spontaneous activities are core parts of Recess.

User-Created Activities

	☰ Task	☰ Deliverable
1	Create activities schema	<code>activities</code> table
2	Create participant schema	<code>activity_participants</code>
3	Implement activity creation	Verified users can create activities
4	Implement activity editing	Creator can update activities
5	Implement activity cancellation	Activity cancellation
6	Implement join requests	Request-to-join workflow
7	Add participant limits	Small-group restriction
8	Add automatic activity expiry	Expired activity removal

9	Add activity RLS policies	Access control
10	Add activity report/block rules	Safety restrictions

Official Events

	☰ Task	☰ Deliverable
1	Create official events schema	<code>official_events</code> table
2	Create RSVP schema	<code>event_rsups</code> table
3	Restrict event creation	Approved organisation accounts only
4	Implement event creation	Event publishing
5	Implement event updates	Event management
6	Implement RSVP workflow	User participation
7	Add event approval status	<code>draft</code> , <code>approved</code> , <code>cancelled</code> , <code>expired</code>
8	Add automatic event expiry	Expired event removal
9	Add event location data	Public venue coordinates
10	Add event RLS policies	Event permissions

Real-Time Map

	☰ Task	☰ Deliverable
1	Define map data format	Standard map payload
2	Create active-map query	Users, activities and events
3	Add area-based filtering	Nearby result filtering
4	Configure Supabase Realtime	Real-time updates
5	Add event/activity channels	Map update subscriptions
6	Prevent exact user-location exposure	Privacy protection
7	Test map permissions	Security test coverage

Suggested Map Data Types

Plain Text ▾

- 1 presence
- 2 activity
- 3 official_event
- 4 public_venue

Activity Rules

- Only verified users can create activities.
- Activities must use a predefined area or public venue.
- Users cannot enter private addresses.
- Activities have a maximum participant limit.
- Activities automatically expire after the end time.
- Blocked users cannot view or join activities.

Exit Criteria

- Users can create small spontaneous activities.
- Approved organisations can publish official events.
- Activities and events appear on the real-time map.
- Expired items automatically disappear.
- Exact user locations are never exposed.

Phase 4 — Matching and Connections

Suggested duration: 5–7 working days

	☰ Task	☰ Deliverable
1	Define matching rules	Matching specification
2	Create matching query/function	Deterministic matching
3	Add time-overlap calculation	Availability compatibility
4	Add area compatibility	Location compatibility
5	Add shared-interest calculation	Interest similarity
6	Exclude blocked users	Safety filtering
7	Exclude existing connections	Duplicate prevention
8	Create suggestions endpoint	Up to three suggestions
9	Create connection requests	Request workflow
10	Add accept/decline/cancel actions	Request management
11	Add connection RLS	Connection privacy
12	Test matching performance	Query and integration tests

Matching Rules

A user can be recommended when:

- Both users are verified;
- They belong to an approved community;
- Their availability overlaps;
- Their approximate areas are compatible;
- They have at least one shared interest;
- There is no block relationship;
- They are not already connected.

Exit Criteria

Users can receive relevant suggestions and establish connections through the backend.

Phase 5 — Chat and Safety

Suggested duration: 5–7 working days

	☰ Task	☰ Deliverable
1	Create messages schema	<code>messages</code> table
2	Implement private chat	Connection-based chat
3	Implement activity chat	Small-group activity chat
4	Configure Realtime messages	Live message delivery
5	Add message RLS	Participant-only access
6	Implement block user	Block workflow
7	Implement report user	Report workflow
8	Add message rate limiting	Spam prevention
9	Add request rate limiting	Abuse prevention
10	Test unauthorised access	Security validation
11	Document moderation workflow	Report-handling guide

Safety Rules

- Only connected users can use private chat.
- Only approved activity participants can use activity chat.
- Users cannot read another user's conversations.
- Blocking must remove map, activity and matching visibility.
- Reports must not be visible to the reported user.
- Exact GPS locations must never be stored for personal presence.

Exit Criteria

Connected users can chat privately, activity participants can coordinate, and users can block or report others.

Phase 6 — Testing, Deployment and Documentation

Suggested duration: 3–5 working days

	☰ Task	☰ Deliverable
1	Run full backend test suite	Test report
2	Audit all RLS policies	Security checklist
3	Test authentication edge cases	Auth test report
4	Test map privacy	Location privacy report
5	Test activity and event expiry	Expiry test report
6	Test matching logic	Matching test report
7	Test blocking and reporting	Safety test report
8	Configure staging environment	Staging deployment
9	Configure production environment	Production setup
10	Add monitoring and logs	Monitoring setup
11	Write backend README	Setup and deployment guide
12	Prepare frontend integration guide	API documentation

Exit Criteria

The backend supports the full MVP flow:

Plain Text ▾

1

Register

2

→ Verify

3

→ Create profile

4

→ Set availability

5

→ View map

6

→ Create or join activity

7

→ Match with users

8

→ Connect

9

→ Chat

10

→ Block/report

9. Backend Task Ownership Summary

	☰ Area	☰ Primary Owner	☰ Supporting Owner
1	Architecture and API contracts	BE Lead	All backend members
2	Database and migrations	BE Data & Security	BE Lead

3	RLS and privacy	BE Data & Security	BE Realtime & QA
4	Authentication	BE Core Features	BE Lead
5	Profiles and availability	BE Core Features	BE Data & Security
6	Activities and events	BE Core Features	BE Data & Security
7	Real-time map	BE Realtime & QA	BE Core Features
8	Matching	BE Core Features	BE Data & Security
9	Chat	BE Core Features	BE Realtime & QA
10	Blocking and reporting	BE Core Features	BE Data & Security
11	Testing and deployment	BE Realtime & QA	BE Lead

10. Backend Deliverables to Frontend

At the end of each phase, the Backend team must provide:

- Database migration;
- API or Supabase query documentation;
- Request and response examples;
- RLS/security rules;
- Error response format;
- Seed data where necessary;
- Test evidence;
- Notes about any limitations.

Frontend should not integrate with undocumented tables or unfinished backend functions.

11. Definition of Done for Backend Tasks

A backend task is complete only when:

- The implementation is committed;
- A migration exists if the database changed;
- RLS policies are implemented;
- Input validation is included;
- Error handling is defined;
- Relevant tests pass;
- API documentation is updated;
- Another backend member has reviewed the pull request;
- Frontend receives the integration details.